Digital Twin of a Cantilever Beam with Rotational Spring (Bolted Joint) Support

Abstract:

A digital twin was developed for a cantilever beam with a bolted joint support, modelled as a rotational spring of stiffness kθ. The governing beam equation was derived with this boundary condition, and a corresponding computational model was built and validated against classical limiting cases. Using measured free-vibration frequencies, an inverse identification algorithm estimated the actual joint stiffness, which was validated through multi-mode consistency and repeatability checks. By progressively loosening the joint, the digital twin tracked the resulting stiffness degradation through a Damage Index, and trend extrapolation was used to forecast future degradation, verified against a physically tested stage. This demonstrates a complete sense–identify–predict loop characteristic of a functioning digital twin system.

What This Project Is About

A cantilever beam is normally assumed to be rigidly fixed at one end. In reality, most supports (bolted joints, welded connections) are never perfectly rigid — they have some small rotational flexibility. This project treats that support as a rotational spring with stiffness kθ, and builds a digital twin that:

1. Measures how the real beam vibrates.
2. Uses that measurement to figure out the actual stiffness of the joint (kθ).
3. Tracks how kθ changes when the joint is loosened (simulating damage).
4. Predicts future degradation before it becomes critical.

In short: the digital twin is a virtual model of the beam that stays synchronized with the real beam by continuously updating itself using vibration data.

Overall Project Flow (Big Picture)

This loop — sense → identify → compare → predict — is the heart of the digital twin. It proceeds in this order:

1. Physical beam is excited; sensor records vibration data.
2. Bolt torque controls the actual (unknown) kθ of the support.
3. Signal processing (FFT) extracts the natural frequency from the recorded signal.
4. An inverse identification algorithm estimates the current kθ from that frequency.
5. The identified kθ is compared against the healthy baseline to compute a Damage Index (DI).
6. DI is tracked across multiple states to predict future trends.
7. The system issues a warning/classification based on the predicted trend.

Summary Table — The Whole Project in One Glance

Phase	What You Do	What You Get
1. Theory	Derive frequency equation with spring boundary condition	Characteristic equation, f(kθ)
2. Model	Code the solver in MATLAB/Python	Validated digital model
3. Rig	Build beam + bolted joint + sensor	Physical test setup
4. Calibration	Match model to measured f1 at full torque	Baseline kθ
5. Validation	Check multi-mode + repeatability	Trust in the model
6. Damage simulation	Loosen bolt in stages, re-identify kθ	DI vs. damage stage table
7. Prediction	Fit trend, extrapolate, verify	Forecast + verification result
8. Demo	Build live dashboard	Final working digital twin

Detailed Phase-by-Phase Steps

PHASE 1: Build the Theory (Do this first, on paper)

Develop the mathematical model following the Hamilton's principle

PHASE 2: Build the Digital (Computer) Model

Develop the MATLAB code for the mathematical model

PHASE 3: Build the Physical Rig

1. Take a thin metal beam (aluminium or mild steel strip)
2. Clamp one end between two plates using bolts — this is your "support." Tightening the bolts more = stiffer joint; loosening = weaker joint (lower kθ).
3. Attach a sensor: an accelerometer (e.g., MPU6050) near the free end connected to Arduino/ESP32, or a strain gauge if available.
4. Set up data logging so you can record the vibration signal (displacement/acceleration vs. time) on a computer.
5. Set up an excitation method: pull the beam tip down slightly and release (free vibration) or give it a light tap.

PHASE 4: Baseline Calibration (Teaching the Twin What "Healthy" Looks Like)

1. Fully tighten the bolts (maximum, consistent torque). This is your baseline/healthy state.
2. Excite the beam and record the free-vibration signal.
3. Apply FFT (Fast Fourier Transform) to the recorded signal to extract the measured natural frequency (f_1).

4. Run an inverse identification routine: guess a kθ, compute the model's predicted f_1, and adjust kθ (using optimization, or simple trial-and-error/bisection) until the model's f_1 matches the measured f_1. The kθ that achieves this match is your identified baseline kθ.

This is the moment the "twin" is born — your virtual model now matches your real beam.

PHASE 5: Validate the Calibration (Very Important — Don't Skip)

You must prove the model isn't just "fitted" by coincidence. Use data you did not use for calibration:

Step 5.1 — Multi-mode check

Use the identified kθ to predict the 2nd and 3rd natural frequencies. Compare these predictions to the actual measured 2nd/3rd frequencies (from the same FFT). A close match means the model is trustworthy.

Step 5.2 — Repeatability check

Repeat the baseline test 3–5 times. The identified kθ should come out nearly the same each time. Report the average and the spread (standard deviation).

PHASE 6: Simulate Damage (Loosening the Joint)

1. Loosen the bolts to a new, lower torque setting (e.g., 75% of baseline). Keep note of this torque value.
2. Repeat the excitation + FFT + inverse identification process (same as Phase 4) at this new state to get a new identified kθ.
3. Repeat for several more loosening stages (e.g., 50%, 25% of baseline torque), each time recording the identified kθ.

Important note about bolted joints

Since bolt torque doesn't translate to a precisely known kθ (due to friction effects), you do NOT have a "true" kθ to compare against here (unlike if you used a torsion spring). Instead, validate your results this way:

- Monotonicity check: identified kθ should consistently decrease as torque decreases. If it jumps around unpredictably, that signals noise or joint slip issues — discuss this honestly in your report.
- Multi-mode consistency check: at each new torque state, confirm the identified kθ also correctly predicts the higher modes measured at that same state.

Build your results table:

State	Torque	Identified kθ	f1 (Hz)	DI (%)
Healthy	100%	k1 (baseline)	f1,1	0%
Damage 1	75%	k2	f1,2	...
Damage 2	50%	k3	f1,3	...
Damage 3	25%	k4	f1,4	...

Compute Damage Index (DI) for each state:

$$DI = [(k\theta,healthy - k\theta,current) / k\theta,healthy] \times 100\%$$

Define a simple status rule, for example:

- DI < 20% → Healthy

- 20–50% → Warning
- > 50% → Critical

PHASE 7: Predict Future Degradation (What Makes It a "Twin," Not Just a Sensor)

1. Plot DI (y-axis) against damage stage number or torque level (x-axis).
2. Fit a simple trend line (linear fit is fine and easiest to justify with limited data points).
3. Extrapolate the trend to predict DI at a stage you have NOT yet physically tested.
4. Predict-then-verify (do this live if possible): announce the prediction, physically set the bolt to that torque, measure the actual DI, and compare prediction vs. actual — report the error.
5. (Optional) Estimate a simple "remaining life" style metric: at the current trend rate, how many more loosening stages until DI crosses your Critical threshold?

This "predict → verify" demonstration is the single strongest proof that your project is a real digital twin and not just a data-logging exercise.

PHASE 8: Package the Final Demo

1. Build a simple live dashboard (Python with Streamlit/Tkinter, or MATLAB GUI) that reads live sensor data, runs FFT automatically, identifies kθ automatically, and displays current DI and status (Healthy/Warning/Critical) on screen.
2. For the viva/demo: physically loosen the bolt in front of the examiners and show the dashboard react in real time. This is the most convincing part of the whole project.